

LC 2858: Minimum Edge Reversals

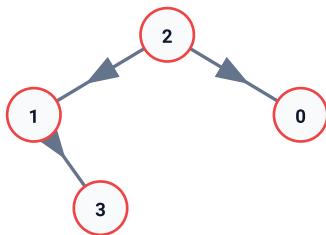
Authored by : Arpit Raj, LNMIIT Jaipur | Date: September 20, 2026

Companies: Meesho, Mathworks

Problem Statement: Minimum Edge reversals so that every node is reachable.

Then we need to output an array where `arr[i]` is min edge reversal required if start from i^{th} node.

Example (n = 4)



Array Outputs

- `arr_0 = 1` , $(2) \rightarrow (0)$ needs to be reversed
- `arr_1 = 1` , $(1) \leftarrow (2)$ needs to be reversed
- `arr_2 = 0` , 2 is already reaching everyone
- `arr_3 = 2` , $(3) \leftarrow (1)$ and $(1) \leftarrow (2)$

Algorithm Notes

S1 build an adjacency list, but store original start node

S2 find `ans[0]` , $\text{curr} \rightarrow \text{next}$ (correct : cost $\rightarrow 0$), $\text{next} \rightarrow \text{curr}$ (x : cost $\rightarrow 1$)

S3 now this will act as base for other nodes, move starting node from curr to next, so $\text{curr} \rightarrow \text{next}$ mean
`ans[next] = ans[curr] + 1`

S4 if edge was $\text{next} \rightarrow \text{curr}$ hence this is valid, as we are treating all future nodes "next" as starting points

C++ Implementation (Part 1)

```
class Solution { public: vector<int> minEdgeReversals(int n,
vector<vector<int>>& edges) { vector<int> ans(n, 0); vector<vector<pair<int,
int>>> adj(n); int x = edges.size(); for(int i = 0; i < x; i++) { int u =
edges[i][0]; int v = edges[i][1]; adj[u].push_back({v, u});
adj[v].push_back({u, u}); } queue<int> q; vector<int> seen(n, 0); int curr;
q.push(0); seen[0] = 1; while(!q.empty()) { curr = q.front(); q.pop();
for(auto &it : adj[curr]) { int next = it.first; int orig = it.second;
if(seen[next]) continue; if(orig == curr) { //curr->next //correct for 0
ans[0] = ans[0] + 0; } else { //next->curr //need to reverse cause curr is 0
for base case ans[0]++; } seen[next] = 1; q.push(next); } }
```

C++ Implementation (Part 2)

```
    seen.assign(n, 0); q.push(0); seen[0] = 1; //now doing for other  
starting positions //eg curr->next me //next is the new start position  
    while(!q.empty()) { curr = q.front(); q.pop(); for(auto &it : adj[curr]) { int  
        next = it.first; int orig = it.second; if(seen[next]) continue; if(orig ==  
        curr) { //curr->next //ab next is the new start pos ans[next] = ans[curr] + 1;  
    } else { //next->curr //incorrect for curr->next //which is already considered  
in ans[curr] //but correct in next-> curr //sp -1 ans[next] = ans[curr] - 1; }  
    seen[next] = 1; q.push(next); } } return ans; } };
```